ZAINA RAHEEN A P

241801328

Learning Objectives

To understand the concept of K-Means Clustering and unsupervised learning. To learn how to perform customer segmentation using Annual Income and Spending Score. To understand the importance of data preprocessing and feature standardization. To determine the optimal number of clusters using the Elbow Method. To visualize and interpret customer clusters and their centroids. To analyze cluster characteristics and suggest suitable marketing strategies.

Built-in Objectives

To implement K-Means Clustering using the scikit-learn library. To use Pandas for loading, exploring, and manipulating the dataset. To use StandardScaler for feature standardization. To use Matplotlib and Seaborn for data and cluster visualization. To calculate cluster centroids, customer counts, average income, and average spending score. To save the clustered dataset for further analysis.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```
from google.colab import files

uploaded = files.upload()
```

```
Choose Files  Mall_Customers.csv
Mall_Customers.csv(text/csv) - 4286 bytes, last modified: 9/29/2026 - 100% done
Saving Mall_Customers.csv to Mall_Customers.csv
```

```
import pandas as pd

df = pd.read_csv("Mall_Customers.csv")

df.head()
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Next steps: Generate code with df New interactive sheet

```
df = pd.read_csv("Mall_Customers.csv")

df.head()
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Next steps: Generate code with df | New interactive sheet

```
print("First 5 records:")
display(df.head())
```

```
First 5 records:
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
print("Last 5 records:")
display(df.tail())
```

```
Last 5 records:
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

```
print("Shape of dataset:", df.shape)
```

```
Shape of dataset: (200, 5)
```

```
print("Data Types:")
print(df.dtypes)
```

```
Data Types:
CustomerID                int64
Genre                    object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object
```

```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
CustomerID                0
Genre                     0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

```
print("Number of duplicate records:")
print(df.duplicated().sum())
```

```
Number of duplicate records:
0
```

```
print("Summary Statistics:")
display(df.describe())
```

Summary Statistics:

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

```
display(X.head())
```

	Annual Income (k$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

```
X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=['Annual Income', 'Spending Score']
)

display(X_scaled_df.head())
```

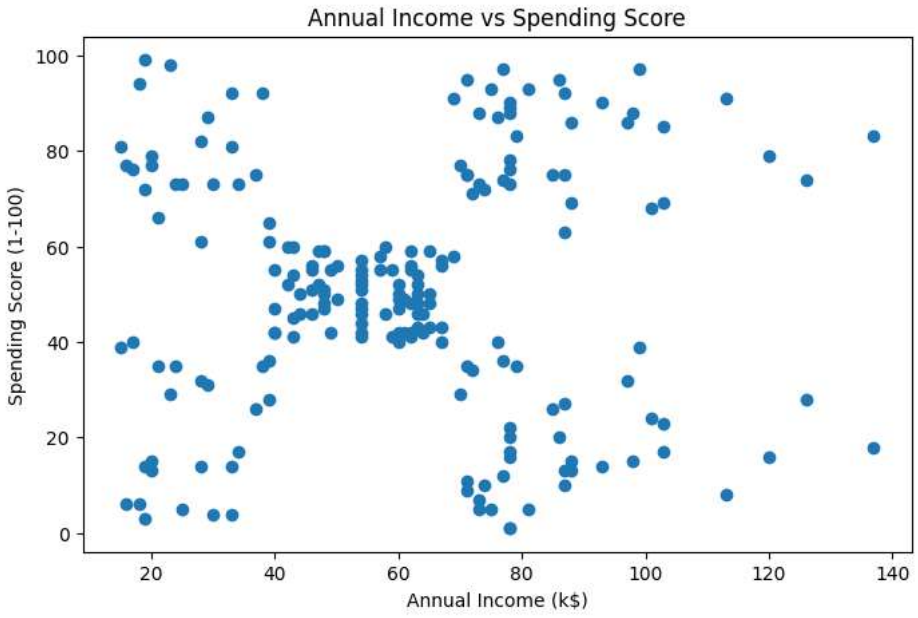
	Annual Income	Spending Score
0	-1.738999	-0.434801
1	-1.738999	1.195704
2	-1.700830	-1.715913
3	-1.700830	1.040418
4	-1.662660	-0.395980

```
plt.figure(figsize=(8, 5))

plt.scatter(
    df['Annual Income (k$)'],
    df['Spending Score (1-100)']
)

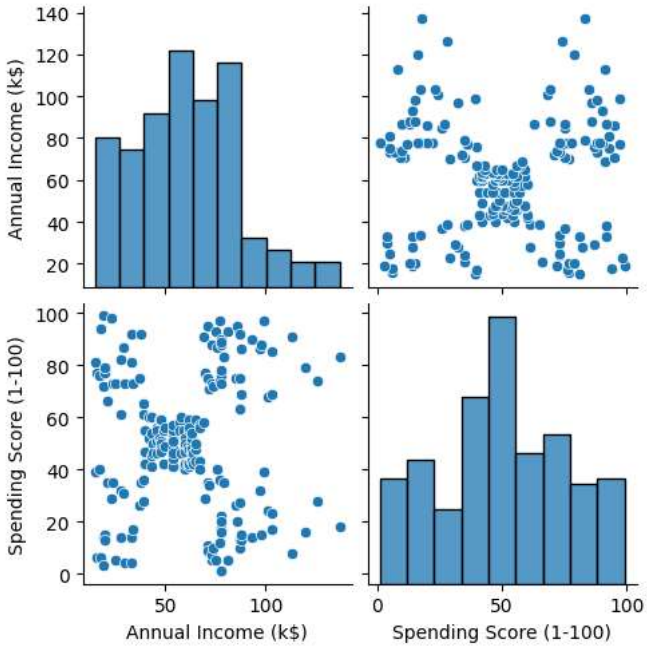
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Annual Income vs Spending Score')

plt.show()
```




```
sns.pairplot(
    df[['Annual Income (k$)', 'Spending Score (1-100)']]
)

plt.show()
```




```
inertia = []

for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)

    inertia.append(kmeans.inertia_)
```

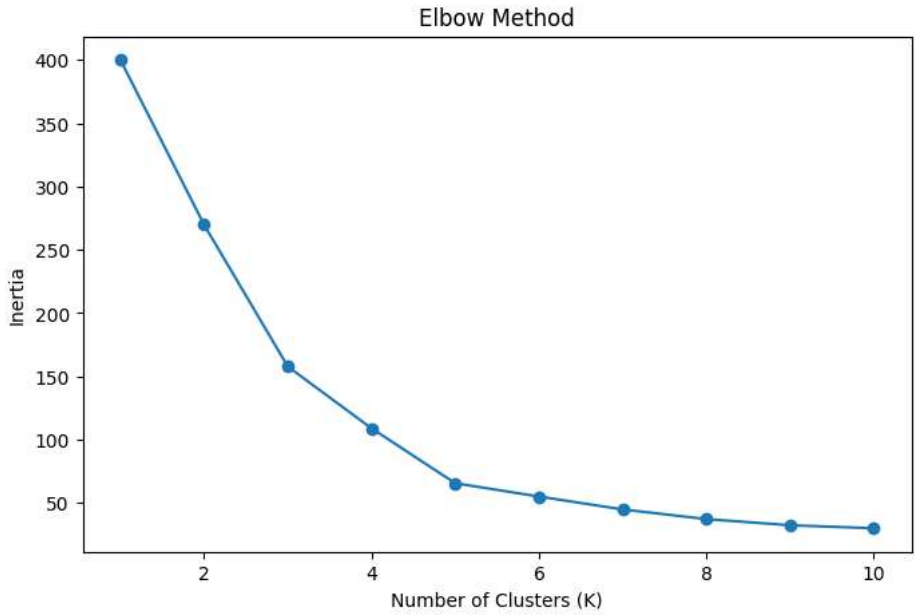
```
plt.figure(figsize=(8, 5))

plt.plot(
```

```
    range(1, 11),
    inertia,
    marker='o'
)

plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method')

plt.show()
```




```
kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(X_scaled)
```

```
df['Cluster'] = clusters
```

```
display(df.head())
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4

```
df.to_csv(
    "Mall_Customers_Clustered.csv",
    index=False
)

print("Clustered dataset saved successfully.")
```

```
Clustered dataset saved successfully.
```

```
plt.figure(figsize=(10, 6))

plt.scatter(
    df['Annual Income (k$)'],
    df['Spending Score (1-100)'],
```

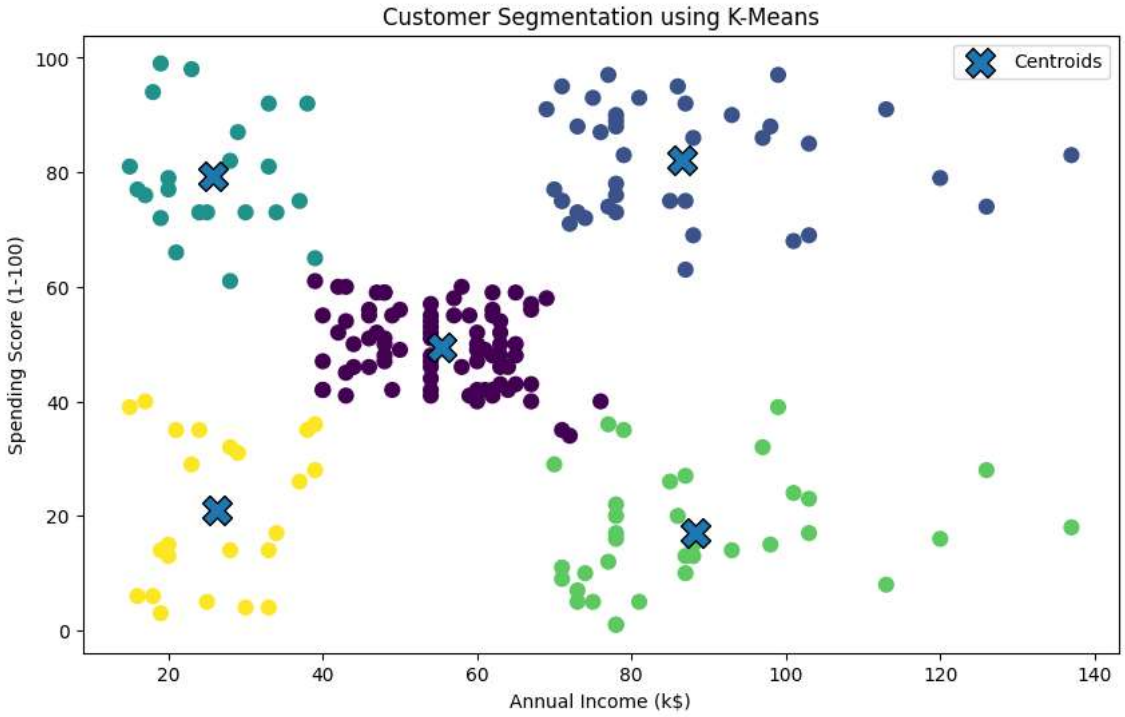
```
    c=df['Cluster'],
    cmap='viridis',
    s=60
)

centroids = scaler.inverse_transform(
    kmeans.cluster_centers_
)

plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker='X',
    s=250,
    edgecolor='black',
    label='Centroids'
)

plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Customer Segmentation using K-Means')

plt.legend()
plt.show()
```




```
centroids = scaler.inverse_transform(
    kmeans.cluster_centers_
)

centroid_df = pd.DataFrame(
    centroids,
    columns=[
        'Annual Income (k$)',
        'Spending Score (1-100)'
    ]
)

print("Cluster Centroids:")
display(centroid_df)
```

Cluster Centroids:

	Annual Income (k$)	Spending Score (1-100)
0	55.296296	49.518519
1	86.538462	82.128205
2	25.727273	79.363636
3	88.200000	17.114286
4	26.304348	20.913043

Next steps: Generate code with centroid_df | New interactive sheet

```
cluster_counts = df['Cluster'].value_counts().sort_index()

print("Number of customers in each cluster:")
print(cluster_counts)
```

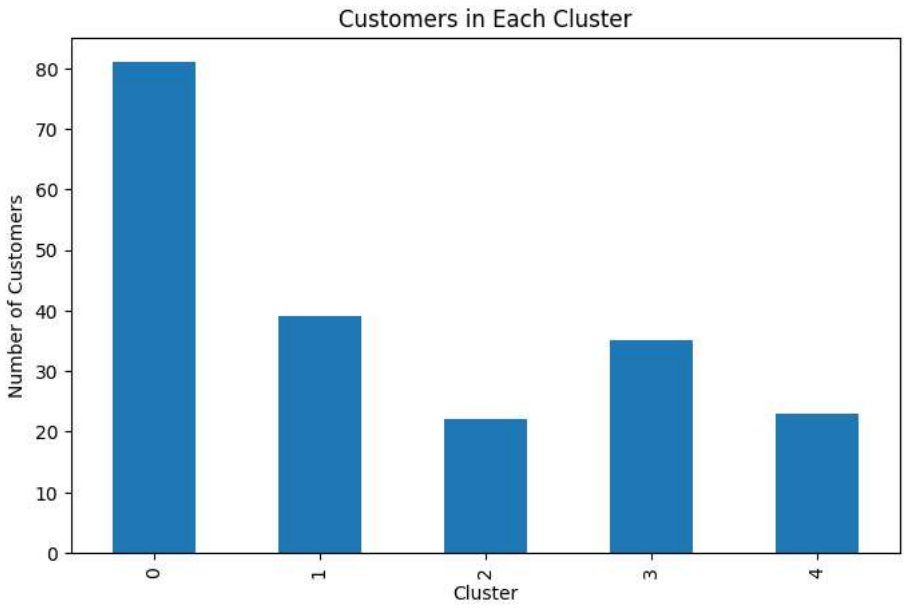
```
Number of customers in each cluster:
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64
```

```
plt.figure(figsize=(8, 5))

cluster_counts.plot(kind='bar')

plt.xlabel('Cluster')
plt.ylabel('Number of Customers')
plt.title('Customers in Each Cluster')

plt.show()
```




```
cluster_summary = df.groupby('Cluster')[
    ['Annual Income (k$)', 'Spending Score (1-100)']
].mean()

display(cluster_summary)
```

Cluster	Annual Income (k$)	Spending Score (1-100)
0	55.296296	49.518519
1	86.538462	82.128205
2	25.727273	79.363636

Next steps: Generate code with cluster_summary | New interactive sheet

```
cluster_summary = cluster_summary.round(2)

display(cluster_summary)
```

Cluster	Annual Income (k$)	Spending Score (1-100)
0	55.30	49.52
1	86.54	82.13
2	25.73	79.36
3	88.20	17.11
4	26.30	20.91

Next steps: Generate code with cluster_summary | New interactive sheet

```
cluster_analysis = df.groupby('Cluster').agg(
    Customer_Count=('Cluster', 'size'),
    Average_Income=('Annual Income (k$)', 'mean'),
    Average_Spending_Score=('Spending Score (1-100)', 'mean')
).round(2)

display(cluster_analysis)
```

Cluster	Customer_Count	Average_Income	Average_Spending_Score
0	81	55.30	49.52
1	39	86.54	82.13
2	22	25.73	79.36
3	35	88.20	17.11
4	23	26.30	20.91

Next steps: Generate code with cluster_analysis | New interactive sheet

Learning Outcomes

After completing this experiment, I will be able to:

Understand the working principle of K-Means Clustering. Preprocess and standardize data before applying a clustering algorithm. Determine an appropriate number of clusters using the Elbow Method. Implement K-Means clustering using Python and scikit-learn. Visualize and interpret different customer segments. Analyze cluster characteristics based on income and spending behavior. Apply clustering results to real-world problems such as customer segmentation and targeted marketing.